

TD/TP 1 — Premiers pas du C vers le C++

Initialisation, références et `const`

MAM4 — Programmation C++ Durée : 2 heures

Objectif. Partir de constructions déjà connues en C pour découvrir trois outils fondamentaux du C++ : l'initialisation sûre, les références et `const`. Les exercices sont courts et progressifs. Il n'est pas nécessaire de tout terminer pour valider la séance : les exercices 1 à 5 constituent le cœur du TD.

Compiler avec :

```
g++ -std=c++20 -Wall -Wextra -pedantic fichier.cpp -o programme
```

Lire les messages du compilateur : dans plusieurs exercices, une erreur de compilation est volontaire.

Les fichiers de départ sont fournis dans l'archive `code-td-seance-01.zip`. Après extraction, `make` compile les programmes ; chaque fichier numéroté correspond à un exercice.

Organisation conseillée. 0–10 min : exercice 1 ; 10–30 min : exercice 2 ; 30–60 min : exercice 3 ; 60–85 min : exercice 4 ; 85–110 min : exercice 5 ; 110–120 min : exercice 6 et dépôt. En cas de difficulté, terminer les exercices 1 à 5 avant de commencer l'exercice 6.

Exercice 1 — Un fichier C n'est pas toujours du C++ (environ 10 min)

Objectif concret. Observer une différence réelle entre les compilateurs C et C++.

```
#include <stdlib.h>

int main(void) {
    int n = 8;
    int* values = malloc(n * sizeof *values);
    if (values == NULL) return 1;
    free(values);
}
```

- Compiler comme du C avec `gcc -std=c17 -Wall -Wextra -pedantic`, puis comme du C++20 avec `g++`. Que dit le compilateur C++ ?
- `malloc` renvoie un `void*`. Ne pas ajouter de conversion uniquement pour faire disparaître l'erreur.
- Observer les deux adaptations suivantes. Dans laquelle devez-vous libérer vous-même la mémoire ?

```
int* values{new int[n]};
// utilisation
delete[] values;
```

```
#include <vector>
std::vector<int> values(n);
// aucune libération explicite
```

À retenir. En C++, on évite généralement `malloc/free` dans le code applicatif et on préfère des objets qui gèrent eux-mêmes leurs ressources, comme `std::vector`.

Exercice 2 — Initialisation et conversions (environ 20 min)

Objectif concret. Lire des réels, effectuer un calcul simple et détecter une perte d'information.

Écrire un programme qui lit deux nombres réels avec `std::cin`, calcule leur moyenne et leur distance avec `std::abs`, puis affiche les résultats avec `std::cout`. Prévoir `<iostream>` et `<cmath>`, et initialiser ainsi :

```
double x{};
double y{};
```

Tester ensuite séparément :

```
int a = 2.9;
int b{2.9};
```

- Quelle ligne compile et quelle ligne est refusée ?
- Quelle information est perdue lors de la conversion de 2.9 vers `int` ?
- Pourquoi ce contrôle est-il utile dans un calcul scientifique ?

À retenir. L'initialisation avec `{}` permet au compilateur de détecter certaines conversions qui perdent de l'information.

Exercice 3 — Découvrir les références (environ 30 min)

Objectif concret. Permettre à une fonction de modifier directement deux résultats.

En C, on passe souvent l'adresse d'une variable lorsqu'une fonction doit la modifier. En C++, une référence permet d'exprimer plus directement cette intention.

Partir de la signature C connue, puis écrire et tester la version C++ :

```
void minmax(const double* values, int n,
           double* minimum,
           double* maximum);
```

```
void minmax(const double* values, int n,
           double& minimum,
           double& maximum);
```

L'appel C++ doit prendre la forme suivante :

```
double lo{};
double hi{};
minmax(values, n, lo, hi);
```

1. La fonction doit-elle modifier `lo` et `hi` ?
2. Doit-elle modifier les valeurs du tableau ?
3. Pourquoi trouve-t-on donc `const` devant `double*`, mais pas devant `lo` et `hi` ?

Tester aussi :

```
double x{3.0};
double& r{x};
r = 7.0;
std::cout << x << '\n';
```

Quelle valeur est affichée ? Que représente `r` par rapport à `x` ?

Mini-exercice — Une fonction qui renvoie une référence.

Compléter la fonction, puis prédire la sortie avant de compiler :

```
#include <iostream>

int& toto(int& value)
{
    // TODO : renvoyer l'objet reçu
}

int main()
{
    int z{4};

    toto(z) = 18;

    std::cout << "z = " << z << '\n';
    std::cout << &z << '\n';
    std::cout << &toto(z) << '\n';
}
```

- a) Pourquoi l'appel `toto(z)` peut-il apparaître à gauche de `=` ?
- b) Combien d'objets de type `int` sont présents ? Que montrent les deux adresses affichées ?
- c) Remplacer provisoirement le type de retour `int&` par `int`. Prédire le résultat, compiler, puis expliquer le diagnostic.

À retenir. Une référence est un autre nom donné à un objet existant. Une fonction peut également renvoyer cet alias.

Exercice 4 — T, T& ou const T& ? (environ 25 min)

Objectif concret. Choisir un paramètre en se demandant si la fonction lit ou modifie l'objet.

```
struct Sample {
    std::string name;
    std::vector<double> values;
};
```

Pour cet exercice, `std::string` et `std::vector` sont fournis comme des types déjà utilisables. Leur fonctionnement sera expliqué à la séance suivante : ici, le seul objectif est de choisir entre `T`, `T&` et `const T&`.

Pour décider : `T` donne à la fonction sa propre valeur ; `T&` lui permet de modifier l'objet existant ; `const T&` lui permet de lire l'objet existant sans le modifier.

Pour chaque opération, demander seulement : la fonction doit-elle lire, modifier ou travailler sur une copie ? Choisir ensuite entre `Sample`, `Sample&` et `const Sample&`.

- a) calculer la moyenne d'un **Sample** sans le modifier ;
- b) centrer sur place toutes ses valeurs ;
- c) afficher un **Sample** sans le modifier ;
- d) calculer le carré d'un **double** ;
- e) multiplier un **double** par deux en modifiant la variable reçue.

Après avoir fait les choix, implémenter et tester les signatures suivantes :

```
double mean(const Sample& s);  
void center(Sample& s);  
void print(const Sample& s);  
double square(double x);  
void multiply_by_two(double& x);
```

À retenir. Pour choisir un paramètre, demander d'abord : la fonction doit-elle modifier l'objet ?

Exercice 5 — Ce code compile-t-il ? (environ 25 min)

Objectif concret. Prédire une erreur simple, compiler, puis expliquer le diagnostic.

Tester les lignes progressivement, en commentant temporairement celles qui bloquent la compilation :

```
double x{2.0};
const double y{3.0};

double& r1{x};
double& r2{y};
const double& r3{y};

r1 += 1.0;
r3 += 1.0;
```

Pour chaque ligne : prédire si elle compile, vérifier, puis expliquer avec des mots simples. On pourra écrire : « **r2** permettrait de modifier une valeur déclarée **const** » ou « **r3** est une référence en lecture seule ».

Si le groupe avance bien, tester aussi `const double& r4{4.0};` sans développer davantage ce cas.

À retenir. `const` signifie ici : cette référence permet de lire la valeur, mais pas de la modifier.

Exercice 6 — 0 ou nullptr ? (environ 10 min, si le temps le permet)

Objectif concret. Distinguer le nombre zéro de l'absence de pointeur.

```
void inspect(int value);
void inspect(double* values);

inspect(0);
inspect(nullptr);
```

Ajouter des définitions qui affichent le nom de la fonction appelée, puis répondre :

1. Quelle fonction est appelée avec 0 ?
2. Quelle fonction est appelée avec `nullptr` ?
3. Qu'exprime plus clairement `nullptr` ?

Remarque facultative : dans du vieux code C ou C++, on rencontre aussi `NULL`.

À retenir. `nullptr` représente explicitement un pointeur nul.

À rendre. Les programmes réalisés pendant la séance, compilant avec `-Wall -Wextra -pedantic`, ainsi que les réponses courtes aux questions. Les exercices 1 à 5 constituent le travail attendu. L'exercice 6 est à faire si le temps le permet. Les petits tests peuvent être regroupés lorsque cela est pertinent.